**INSTITUTO POLITÉCNICO DE SANTARÉM**
Escola Superior de Gestão e Tecnologia

# ARTEFACTOS DE GESTÃO DE PROJETO

## Sistema de Gestão Hoteleira (SGH)

*Deliverable 4*




**Autores:**
Guilherme Caçador - 240000675
Rodrigo Silva - 240000762

Engenharia de Software | Docente: Pedro Martins
2025/2026

# 1. Metodologia de Desenvolvimento

O desenvolvimento do SGH seguiu a metodologia ágil Scrum, adaptada ao contexto académico de uma equipa de 2 elementos. Esta abordagem permitiu entregas incrementais, feedback contínuo e adaptação às mudanças de requisitos ao longo do projeto.

## 1.1. Adaptações ao Scrum

- Sprints de 2 semanas: Duração adequada ao calendário académico
- Daily Standups: Substituídos por reuniões síncronas 2-3 vezes por semana
- Product Owner: Papel assumido alternadamente pelos membros da equipa
- Retrospetivas: Realizadas no final de cada sprint para melhoria contínua

## 1.2. Ferramentas Utilizadas

- Trello: Gestão visual do backlog e tarefas (quadro Kanban)
- GitHub: Controlo de versões, code review e CI/CD
- Discord: Comunicação síncrona e assíncrona da equipa
- Draw.io: Criação de diagramas UML colaborativamente

# 2. Product Backlog

O Product Backlog contém todas as User Stories priorizadas, seguindo o formato: 'Como [papel], quero [funcionalidade] para que [benefício]'. Cada história inclui critérios de aceitação, estimativa em story points e estado de implementação.

| ID | User Story | Priorid. | SP | Sprint | Estado |
|---|---|---|---|---|---|
| US-001 | Como rececionista, quero criar uma reserva para que o hóspede tenha o quarto garantido. | Alta | 8 | 1 | Done |
| US-002 | Como rececionista, quero cancelar uma reserva para que o quarto fique disponível. | Alta | 5 | 1 | Done |
| US-003 | Como rececionista, quero consultar disponibilidade para informar o cliente. | Alta | 5 | 1 | Done |
| US-004 | Como rececionista, quero fazer check-in para que o hóspede possa ocupar o quarto. | Alta | 8 | 2 | Done |
| US-005 | Como rececionista, quero fazer check-out e emitir fatura para o hóspede pagar. | Alta | 8 | 2 | Done |
| US-006 | Como administrador, quero gerir o inventário de quartos para manter dados atualizados. | Alta | 5 | 1 | Done |
| US-007 | Como sistema, quero calcular preços sazonais para que as tarifas reflitam a procura. | Média | 8 | 2 | Done |
| US-008 | Como governanta, quero atualizar o estado do quarto para que a receção saiba. | Média | 5 | 2 | Done |
| US-009 | Como técnico, quero reportar avarias para que o quarto seja bloqueado. | Média | 5 | 2 | Done |
| US-010 | Como gestor, quero ver relatórios de ocupação para tomar decisões estratégicas. | Média | 5 | 3 | Done |
| US-011 | Como utilizador, quero autenticar-me para aceder às funcionalidades do meu perfil. | Alta | 5 | 1 | Done |
| US-012 | Como hóspede, quero receber email de confirmação para ter comprovativo da reserva. | Baixa | 3 | 3 | Done |

*Tabela 1: Product Backlog Completo*

## 2.1. Detalhes das User Stories Principais

**US-001: Criar Reserva**

Como rececionista, quero criar uma reserva para que o hóspede tenha o quarto garantido nas datas pretendidas.

**Critérios de Aceitação:**

- Utilizador pode selecionar datas de entrada e saída
- Sistema mostra apenas quartos disponíveis para o período
- Preço total é calculado automaticamente com base na época
- Sistema impede overbooking (exceto admin)
- Reserva fica com estado CONFIRMADA após criação

**US-004: Realizar Check-in**

Como rececionista, quero fazer check-in para que o hóspede possa ocupar o quarto.

**Critérios de Aceitação:**

- Sistema valida que o quarto está no estado PRONTO
- Sistema regista documento de identificação do hóspede (SEF)

- Quarto muda para estado OCUPADO
- Data/hora de check-in é registada

**US-007: Preços Sazonais (Strategy Pattern)**

Como sistema, quero calcular preços sazonais para que as tarifas reflitam automaticamente a procura da época.

**Critérios de Aceitação:**

- Época Alta (Jun-Set): multiplicador 1.5x
- Época Média (Dez-Fev): multiplicador 1.2x
- Época Baixa (restantes): multiplicador 1.0x
- Novas estratégias podem ser adicionadas sem modificar código existente

# 3. Planeamento de Sprints

O projeto foi dividido em 3 sprints de 2 semanas cada, com foco progressivo desde a infraestrutura base até à qualidade e documentação final.

## 3.1. Sprint 1: Fundações (Semanas 1-2)

Objetivo: Estabelecer a estrutura base do projeto, configurar ambiente de desenvolvimento e implementar funcionalidades nucleares de reservas.

**User Stories:**

US-001 (Criar Reserva), US-002 (Cancelar Reserva), US-003 (Consultar Disponibilidade), US-006 (Inventário Quartos), US-011 (Autenticação)

**Entregáveis:**

- Documento de Requisitos (versão inicial)
- Diagramas UML: Casos de Uso, Classes (versão inicial)
- Estrutura Maven configurada com dependências
- Entidades de domínio: Quarto, Reserva, Hospede
- ServicoReserva com operações CRUD básicas

Velocity: 28 Story Points

## 3.2. Sprint 2: Regras de Negócio (Semanas 3-4)

Objetivo: Implementar lógica de negócio complexa, aplicar padrões de desenho e desenvolver módulos de check-in/out, housekeeping e manutenção.

**User Stories:**

US-004 (Check-in), US-005 (Check-out/Fatura), US-007 (Preços Sazonais), US-008 (Estado Limpeza), US-009 (Avarias)

**Entregáveis:**

- Padrão Strategy implementado (ServicoPrecos)
- Padrão Observer implementado (Notificações)
- Padrão Factory implementado (FabricaQuarto, FabricaFatura)
- Diagramas de Sequência (5 cenários)
- Documento de Arquitetura (versão inicial)

Velocity: 34 Story Points

## 3.3. Sprint 3: Qualidade e Finalização (Semanas 5-6)

Objetivo: Garantir qualidade através de testes extensivos, implementar funcionalidades secundárias e preparar toda a documentação final.

**User Stories:**

US-010 (Relatórios), US-012 (Email Confirmação), Testes e Documentação

**Entregáveis:**

- Suite completa de testes JUnit (36+ casos, >70% cobertura)
- Documentação de Testes completa
- Diagramas de Atividade (3 processos)
- Documentação final revista e completa
- Apresentação PowerPoint
- Correção de bugs e refatorização

Velocity: 8 Story Points + Documentação

| Sprint | Foco Principal | Entregáveis Chave | Story Points | Estado |
|---|---|---|---|---|
| Sprint 1 | Fundações e Requisitos | Requisitos, UML inicial, CRUD Reservas | 28 SP | Concluído |
| Sprint 2 | Regras de Negócio | Padrões Design, Check-in/out, Arquitetura | 34 SP | Concluído |
| Sprint 3 | Qualidade e Docs | Testes JUnit, Docs finais, Apresentação | 8 SP | Concluído |

*Tabela 2: Resumo do Planeamento de Sprints*

# 4. Estrutura do Repositório Git

O repositório Git segue uma estrutura profissional com organização clara de código, testes e documentação. Utilizamos Git Flow simplificado com branch main para código estável e branches de feature para desenvolvimento.

## 4.1. Estrutura de Pastas

```
sistema-gestao-hoteleira/
├── .gitignore
├── pom.xml
├── README.md
│
├── docs/
│   ├── requirements.pdf
│   ├── architecture.pdf
│   ├── test-plan.pdf
│   ├── project-management.pdf
│   └── uml-diagrams/
│       ├── use-case-diagram.png
│       ├── class-diagram.png
│       ├── sequence-reserva.png
│       ├── sequence-checkin.png
│       ├── sequence-checkout.png
│       ├── sequence-limpeza.png
│       ├── sequence-manutencao.png
│       ├── activity-reserva.png
│       ├── activity-limpeza.png
│       └── activity-faturacao.png
│
└── src/
    ├── main/
    │   ├── java/
    │   │   └── com/sgh/
    │   │       ├── SghApplication.java
    │   │       ├── controlador/
    │   │       ├── modelo/
    │   │       │   └── enums/
    │   │       ├── repositorio/
    │   │       ├── servico/
    │   │       ├── fabrica/
    │   │       ├── estrategia/
    │   │       ├── observador/
    │   │       └── excecao/
    │   │
    │   └── resources/
    │       └── application.properties
    │
```

```
└── test/
    └── java/
        └── com/sgh/
            ├── servico/
            ├── estrategia/
            └── integracao/
```

## 4.2. Convenções de Commit

Seguimos convenções de commit semânticas para manter histórico claro e rastreável:

- feat: nova funcionalidade (ex: feat: implementar cancelamento de reserva)
- fix: correção de bug (ex: fix: corrigir cálculo de preço sazonal)
- test: adição ou alteração de testes (ex: test: adicionar testes para Strategy)
- docs: alterações na documentação (ex: docs: atualizar diagrama de classes)
- refactor: refatorização sem alterar funcionalidade

## 4.3. Exemplos de Commits do Projeto

```
abc1234 feat: implementar ServicoReserva com CRUD completo
def5678 feat: adicionar padrão Strategy para cálculo de preços
ghi9012 feat: implementar padrão Observer para notificações
jkl3456 feat: criar FabricaQuarto e FabricaFatura
mno7890 test: adicionar testes unitários para ServicoReserva
pqr1234 test: implementar testes de integração com H2
stu5678 fix: corrigir validação de datas em reserva
vwx9012 docs: adicionar diagrama de sequência check-in
yza3456 refactor: extrair lógica de validação para classe própria
```

## 4.4. Contribuições da Equipa

| Membro | Commits | Linhas | Áreas Principais |
|---|---|---|---|
| Guilherme Caçador | 47 | +2,341 / -892 | Backend, Testes |
| Rodrigo Silva | 52 | +2,567 / -743 | Arquitetura, Docs |

*Tabela 3: Contribuições Git por Membro*